perplexity

Can you create a UI for the above code where user will provide test url and business logic and the test result will appear?Provide me a plan first.No code.

Yes. Build a simple web app where the user enters a **test URL** and the **business logic**, clicks **Generate / Run Tests**, and sees generated test scenarios and their results. The safest design separates the UI, backend, LLM, and test execution so the LLM never directly controls the browser or backend. [1] [2]

Overall plan

```
User
  ↓ enters URL + business logic
Frontend UI
  ↓ sends request
Backend API
  ↓ validates inputs
LLM service
  ↓ returns structured test scenarios
Backend test runner
  ↓ opens URL / runs checks
Results store
  ↓
Frontend results table
```

The UI should collect inputs, show progress, display generated cases, and render pass/fail results. The backend—not the browser—should validate inputs, call the LLM, execute tests, and store results. [3]

UI screens

1. Test setup screen

Include these fields:

Field	Purpose
Test URL	The website or API endpoint to test
Business logic	Plain-English rule, such as “Coupon applies only when order amount is greater than 1000”
Test type	UI test, API test, or form-validation test

Field	Purpose
Environment	Optional: staging, production, local
Generate button	Starts scenario generation
Run tests button	Executes the generated scenarios

Add helpful placeholders and validation messages. For example, require a valid URL format and a non-empty business-logic description.

2. Generated scenarios screen

After the LLM responds, display a table:

ID	Scenario	Input / action	Expected result	Status
TC001	Valid amount	1200	Coupon applies	Pending
TC002	Boundary amount	1000	Coupon does not apply	Pending
TC003	Empty amount	Empty	Coupon does not apply	Pending

Let the user review, edit, disable, or delete scenarios before running them. This human-review step is important because LLM-generated tests may misunderstand the rule.[4]

3. Test results screen

Show results after execution:

ID	Scenario	Input	Expected	Actual	Result
TC001	Valid amount	1200	Coupon applies	Coupon applied	Pass
TC002	Boundary amount	1000	Coupon does not apply	Coupon applied	Fail
TC003	Empty amount	Empty	Coupon does not apply	Validation error	Pass

Include a summary at the top: total tests, passed, failed, and execution time.

Core features

- **URL validation:** Check that the URL is well formed and restrict it to approved domains if needed.
- **Business-logic input:** Use a large text box with a character limit.
- **LLM generation:** Convert the business rule into structured JSON test cases.
- **Scenario review:** Let the user inspect and edit generated cases before execution.
- **Test execution:** Use a controlled test runner, such as Playwright for browser tests or HTTP requests for API tests.
- **Result display:** Show pass/fail, actual behavior, screenshots, error messages, and execution time.

- **History:** Save past runs so users can compare results over time.
- **Export:** Allow download of results as CSV, JSON, or PDF.

Backend flow

1. Receive the URL and business logic from the UI.
2. Validate the URL and limit the business-logic length.
3. Build a strict prompt that asks only for JSON.
4. Call the LLM with a JSON schema or structured-output setting.
5. Parse and validate the returned JSON.
6. Show generated scenarios to the user for approval.
7. Execute only approved scenarios in the test runner.
8. Save and return the results.

Use structured output and deterministic validation because the LLM response must be treated as untrusted data, not as instructions. [2] [5] [1]

Security plan

- Never expose the LLM API key in the frontend; keep it only in the backend.
- Validate and limit all user inputs before sending them to the LLM.
- Do not let the LLM choose arbitrary URLs or execute arbitrary commands.
- Use an allowlist for permitted domains, especially if the app can access internal systems.
- Sanitize displayed results to prevent HTML/JavaScript injection.
- Require user approval before running tests against production systems.
- Log test runs and errors, but do not log API keys or sensitive business data.

Prompt injection is a real risk: user-entered business logic or content fetched from a URL could try to alter the LLM's behavior. Treat both as untrusted, separate instructions from data, and validate outputs before execution. [5] [1] [2]

Suggested MVP

For the first version, keep it focused:

1. One page with URL and business-logic fields.
2. A "Generate Test Scenarios" button.
3. A table of generated scenarios.
4. A "Run Tests" button.
5. A results table with pass/fail status.
6. A downloadable JSON/CSV report.

Do not begin with autonomous browsing, self-healing tests, or automatic production execution. Start with a controlled, reviewable workflow, then add automation once the generated scenarios are reliable.[6] [4]

⁂

1. https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
2. https://cheatsheetseries.owasp.org/cheatsheets/AI_Agent_Security_Cheat_Sheet.html
3. https://www.manning.com/preview/build-ai-enhanced-web-apps/chapter-1
4. https://www.cs.ru.nl/bachelors-theses/2025/Dylan_Elens___1051438___An_LLM-Based_End-to-End_Testing_Agent_for_Web_Applications.pdf
5. https://genai.owasp.org/llmrisk/llm01-prompt-injection/
6. https://dl.acm.org/doi/10.1145/3793654.3793743
7. https://cheatsheetseries.owasp.org/cheatsheets/Secure_Coding_with_AI_Cheat_Sheet.html
8. https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
9. https://arxiv.org/html/2504.15546v1
10. https://genai.owasp.org/llmrisk2023-24/llm01-24-prompt-injection/
11. https://github.com/OWASP/AISVS/blob/8906b2be8341abbf4dc43411374681fb81e6ddc6/1.0/research/chapters/C02-User-Input-Validation/C02-01-Prompt-Injection-Defense.md
12. https://github.com/aiminnovations/owasp-CheatSheetSeries/blob/master/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.md
13. https://cheatsheetseries.owasp.org/cheatsheets/Secure_AI_Model_Ops_Cheat_Sheet.html
14. https://cheatsheetseries.owasp.org/cheatsheets/Injection_Prevention_Cheat_Sheet
15. https://cheatsheetseries.owasp.org/cheatsheets/MCP_Security_Cheat_Sheet.html